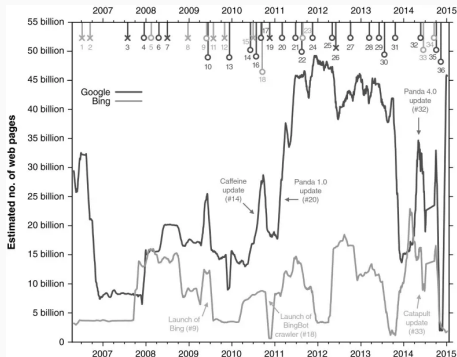


Implémentation de systèmes de gestion de bases de données

Qualité dans les SGBD : contraintes et cohérence

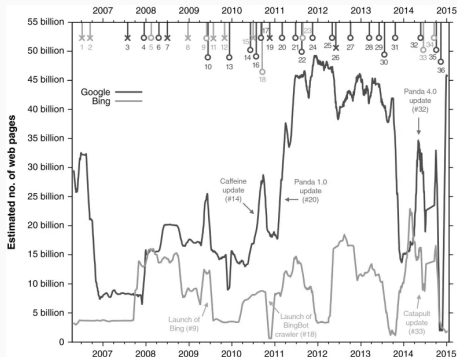
Taille versus qualité : le dilemme des moteurs de recherche

Contrairement à ce que l'on croit la taille de l'index de moteur de recherche (Google, Bing, etc) n'est pas croissante.



Taille versus qualité : le dilemme des moteurs de recherche

Contrairement à ce que l'on croit la taille de l'index de moteur de recherche (Google, Bing, etc) n'est pas croissante.



Comment les SGDB permettent d'assurer la qualité de leurs données ?

Les bases de données relationnelles

Une base de données relationnelles comporte :

- un **schéma** : la structure des tables/reliations (le nom et le type des colonnes/attributs)
- un **contenu** : les lignes stockées dans chaque table
- des **contraintes** d'intégrité assurent la cohérence des données

Exemple

- schéma

Somme(client, total)

Panier(client, produit)

Produit(id, nom, prix)

- contenu

Somme		Panier		Produit		
client	total	client	produit	id	nom	prix
0	10	0	0	0	pommes	5
1	5	0	1	1	poires	5
		1	0	2	carottes	2

Contraintes d'intégrité

Les contraintes d'intégrité permettent de définir des limitations sur le contenu de la base de données. Il y a deux types de contraintes :

- les **contraintes explicite**, qui sont stockés dans le SGBD et peuvent être vérifiées automatiquement à chaque modification du contenu
- les **contraintes implicites**, qui sont dans la tête de l'administrateur de la base de données uniquement

Exemple de contraintes

- une explicite : `Produit.prix >= 0`
- une implicite : le total d'un panier est la somme des prix des objects qu'il contient

Contraintes explicites

Parmi les contraintes d'intégrité qui peuvent être définies dans un SGBD, on trouve :

- les contraintes de clé primaire (clause `PRIMARY KEY`)
- les contraintes référentielles (clause `FOREIGN KEY`)
- les contraintes `CHECK`
- les contraintes définies par des `triggers`.

Contraintes de clés

Soit K un sous-ensemble des colonnes d'une table T ,

- K est une **super-clé** de T , si le nombre de ligne ayant les mêmes valeurs sur les colonnes de K est au plus 1
- K est une **clé candidate** de T , si c'est une super-clé qui contient un nombre minimale de colonnes
- une **clé primaire** de T est une clé candidate de T choisie pour identifier chaque ligne de T

Produit		
id	nom	prix
0	pommes	5
1	poires	5
2	carottes	2

- Quels sont les super-clés, clés candidates ?
- Quel est le choix naturel pour la clé primaire ?

Contraintes de clés

Soit K un sous-ensemble des colonnes d'une table T ,

- K est une **super-clé** de T , si le nombre de ligne ayant les mêmes valeurs sur les colonnes de K est au plus 1
- K est une **clé candidate** de T , si c'est une super-clé qui contient un nombre minimale de colonnes
- une **clé primaire** de T est une clé candidate de T choisie pour identifier chaque ligne de T

Produit		
id	nom	prix
0	pommes	5
1	poires	5
2	carottes	2

- Quels sont les super-clés, clés candidates ?
- Quel est le choix naturel pour la clé primaire ?

On note cela avec `Produit(id, nom, prix)`

Exemple de clé primaire

Panier		Produit		
client	produit	<u>id</u>	nom	prix
0	0	0	pommes	5
0	1	1	poires	5
1	0	2	carottes	2

1. Quelle clé primaire peut-on choisir pour la table Panier ?

Exemple de clé primaire

Panier		Produit		
client	produit	<u>id</u>	nom	prix
0	0	0	pommes	5
0	1	1	poires	5
1	0	2	carottes	2

1. Quelle clé primaire peut-on choisir pour la table Panier ?
{client, produit}
2. Avec cette clé primaire, un client peut-il avoir plusieurs fois le même produit dans son panier ?

Exemple de clé primaire

Panier		Produit		
client	produit	<u>id</u>	nom	prix
0	0	0	pommes	5
0	1	1	poires	5
1	0	2	carottes	2

1. Quelle clé primaire peut-on choisir pour la table Panier ?
{client, produit}
2. Avec cette clé primaire, un client peut-il avoir plusieurs fois le même produit dans son panier ?
3. Comment résoudre ce problème ?

Exemple de clé primaire

Panier		Produit		
client	produit	<u>id</u>	nom	prix
0	0	0	pommes	5
0	1	1	poires	5
1	0	2	carottes	2

1. Quelle clé primaire peut-on choisir pour la table Panier ?
{client, produit}
2. Avec cette clé primaire, un client peut-il avoir plusieurs fois le même produit dans son panier ?

3. Comment résoudre ce problème ?	Panier		
	<u>client</u>	<u>produit</u>	quantité
	0	0	2
	0	1	1
	1	0	4

Les clés étrangères

Une **clé étrangère** est un ensemble de colonnes K d'une table faisant référence à la clé primaire d'une autre table. La clé étrangère est satisfaite si les tuples de valeurs présents sur les colonnes de K sont aussi présents sur les colonnes de la clé primaire référencée.

Panier		Produit		
client	produit	<u>id</u>	nom	prix
0	0	0	pommes	5
0	1	1	poires	5
1	0	2	carottes	2

On souhaite définir une clé étrangère depuis la colonne `produit` de la table `Panier` vers la clé primaire de la table `Produit` i.e. : $\forall t \in \text{Panier}, \exists p \in \text{Produit}, t.\text{produit} = p.\text{id}$

Cette contrainte assure que chaque produit dans un panier possède un nom et un prix.

Une contrainte CHECK permet de restreindre les valeurs prises par les attributs d'un tuple dans une table.

```
CREATE TABLE Somme (  
    client integer PRIMARY KEY,  
    somme integer CHECK (somme > 0)  
);
```

Cohérence d'une base de données

Une base de données est **cohérente** si son contenu vérifie :

- les **contraintes explicites**
- les **contraintes implicites**

On souhaite que tous les modifications apporter à une base de données conservent la cohérence de la base.

Exemple

Somme		Panier		Produit		
<u>client</u>	total	<u>client</u>	<u>produit</u>	<u>id</u>	nom	prix
0	10	0	0	0	pommes	5
1	5	0	1	1	poires	5
		1	0	2	carottes	2

Les opérations sur les bases de données

SQL est le langage standard pour accéder et modifier l'instance d'une base de données relationnelle :

- requêtes d'interrogation

```
SELECT DISTINCT Pa.client, Pr.nom  
FROM Panier AS Pa, Produit AS Pr  
WHERE Pa.produit = Pr.id;
```

- requêtes de mise à jour ou d'insertion

```
UPDATE Produit SET prix = 1.1*prix WHERE nom = 'pommes';
```

- opérations de modification du schéma ou des contraintes

```
CREATE TABLE Produit ( ...
```

Exercices

On souhaite créer le schéma des tables :

- `Panier(client, produit)`
- `Produit(produit, nom, prix)`.

avec les contraintes explicites suivantes :

- `(client, produit)` forme la clé primaire de `Panier`
- `produit` est la clé primaire de `Produit`
- il y a une clé étrangère de `Panier` vers `Produit`
- les prix sont des valeurs positives

Questions

- Écrire les requêtes SQL pour créer ces deux tables.
- Écrire une requête qui retourne le prix total du panier de chaque client.

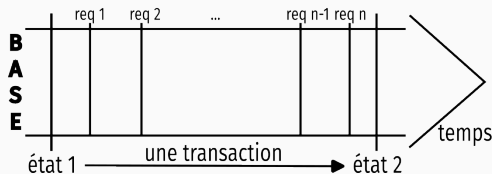
Les transactions

Transactions

Le concept de requête ne permet pas de capturer les opérations atomiques nécessaire à la gestion d'une base de données, c'est pourquoi on a besoin des transactions :

Définition

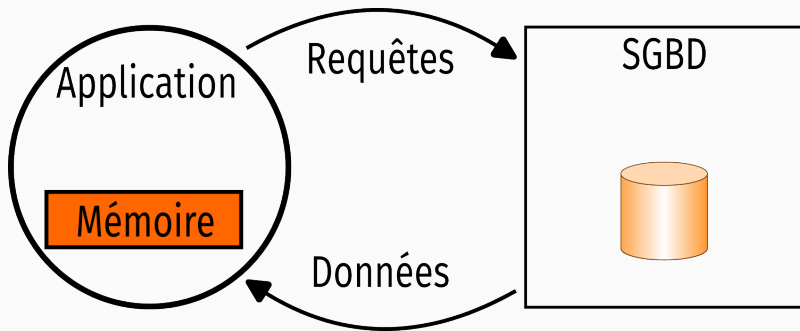
Une transaction est l'exécution d'un programme exécutant plusieurs requêtes sur la base



Exemple de transaction

1. Une valeur x est lue depuis la base de données
2. Un programme est exécuté en prenant x pour entrée et retourne y
3. La valeur y est insérée dans la base de donnée

Boucle transactionnelle



Commandes SQL pour les transactions

SQL permet de définir un groupe de requêtes comme une transaction :

```
-- état 1
-- début de la transaction
BEGIN;
requete 1
requete 2
...
COMMIT; -- commande pour demander la validation de la transaction
-- état 2
```

Pour annuler la transaction, il faut utiliser **ROLLBACK** au lieu de **COMMIT**

Autocommit

Avec l'autocommit, chaque requête envoyée est comme entourée par **BEGIN** et **COMMIT**. Chaque requête est dans une transaction différente.

Les transactions comme actions cohérentes

Somme		Panier		Produit		
		client	produit	id	nom	prix
client	total	0	0	0	pommes	5
0	10	0	1	1	poires	5
1	5 +2	1	0	2	carottes	2
		1	2			

La transaction pour “Le client 1 ajoute des carottes à son panier” est composée de deux actions inséparables :

1. ajouter la ligne correspondante dans Panier
2. ajouter le prix au total dans Somme

Cette transaction permet de vérifier la contrainte implicite sur le total

Exercice sur la rédaction de transactions

Somme		Panier		Produit		
client	total	client	produit	id	nom	prix
0	10	0	0	0	pommes	5
1	5	0	1	1	poires	5
		1	0	2	carottes	2

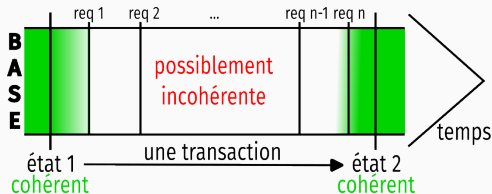
Écrire des transactions pour chacune des manipulations suivantes (on suppose que l'identifiant des produits est connus) :

1. Le client d'identifiant 0 ajoute des carottes à son panier
2. Le prix des pommes augmentent de 1€
3. Les poires ne sont plus disponibles : supprimer les des produits et des paniers

On utilisera un pseudo code avec des variables pour stocker des valeurs lues depuis la base avec : `SELECT attribut INTO variable FROM ...`

Principe de cohérence des transactions

Si une transaction s'exécute en l'absence d'autres transactions ou d'erreur de système sur une base de données cohérente, alors la base de données à la fin de l'exécution est aussi cohérente



Remarques

1. la base de données peut être incohérente au cours d'une transaction
2. Que se passe-t-il s'il y a plusieurs transactions en même temps ?
3. Que se passe-t-il si une transaction est interrompue (ex : le système plante au milieu d'une transaction) ?

Les transactions comme des transformations valides

Au delà de la cohérence, les transactions définissent les transformations valides de la base

Exemple

Compte(num, solde)

solde > 0

Transfert de 100€ entre le compte 123 et 456, une transaction en deux requêtes inséparables :

1. ajouter 100€ au compte 456

```
UPDATE Compte SET solde = solde + 100 WHERE num = 456;
```

2. retirer 100€ au compte 123

```
UPDATE Compte SET solde = solde - 100 WHERE num = 123;
```

Appliquer une seule des deux requêtes :

- peut laisser la base de données cohérente
- crée ou supprime de l'argent : ce n'est pas un transfert bancaire valide !

En résumé

- les transactions sont des programmes qui exécutent des requêtes sur la base
- les transactions permettent de définir des transformations de la base qui conservent sa cohérence
- les transactions permettent de définir des transformations valides d'une base
- la cohérence aux travers des transactions est en partie la responsabilité du programmeur d'applications

La prochaine fois

- les propriétés ACID des transactions dans les SGDB
- les problèmes liés à la concurrence des transactions

Propriétés ACID

On a vu que :

- Les transactions permettent de conserver la cohérence/qualité de la base de données
- Ceci est la responsabilité du programmeur d'applications et des contraintes d'intégrité explicites de la base

Facile à gérer pour le SGBD à condition que :

1. les transactions s'exécutent les unes après les autres (pas de concurrences)
2. une transaction terminent toujours par un succès.

Les transactions doivent vérifier les propriétés **ACID** :

- **Atomicité** chaque transaction s'exécute entièrement ou pas du tout
- **Cohérence** quelque soit l'exécution des transactions, la base de données reste cohérente
- **Isolation** les transactions doivent modifier la base comme si chacune était exécutée seule
- **Durabilité** les modifications apportées par une transaction validée ne sont jamais perdues

Atomicité : tout ou rien

Une transaction n'a que trois états :

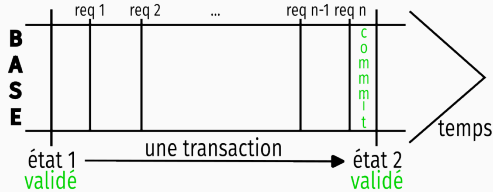
- active, en cours d'exécution
- validée (**committed**), terminée par un succès (après **COMMIT**)
- annulée (**aborted**), terminée par un échec

L'annulation d'une transaction peut être provoquée par :

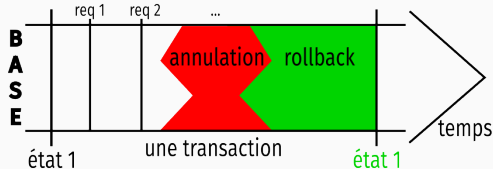
- un plantage (coupure de courant, disque plein, etc)
- le système, si une erreur survient au cours de l'exécution
- l'utilisateur, s'il annule la transaction avec **ROLLBACK**

Atomicité, deux issues possibles : validation ou annulation

Transaction validée



Transaction annulée



Exemples d'annulation de transaction

Quand une transaction est annulée, il faut revenir en arrière et défaire les modifications réalisées : faire un **rollback**.

```
BEGIN;  
  
UPDATE Compte  
SET solde = solde + 100  
WHERE num = 456;
```

💣 le système plante entre les deux requêtes

```
UPDATE Compte  
SET solde = solde - 100  
WHERE num = 123;  
  
COMMIT;
```

1. situation initiale

num	solde
123	100
456	200

2. exécution de la première requête

num	solde
123	100
456	300

3. plantage, le système est éteint

4. au redémarrage, retour en arrière

num	solde
123	100
456	200

Exemples d'annulation de transaction

Quand une transaction est annulée, il faut revenir en arrière et défaire les modifications réalisées : faire un **rollback**.

```
BEGIN;  
  
UPDATE Compte  
SET solde = solde + 100  
WHERE num = 456;
```

❌ la requête viole la contrainte d'intégrité
solde < 250

```
UPDATE Compte  
SET solde = solde - 100  
WHERE num = 123;  
  
COMMIT;
```

1. situation initiale

num	solde
123	100
456	200

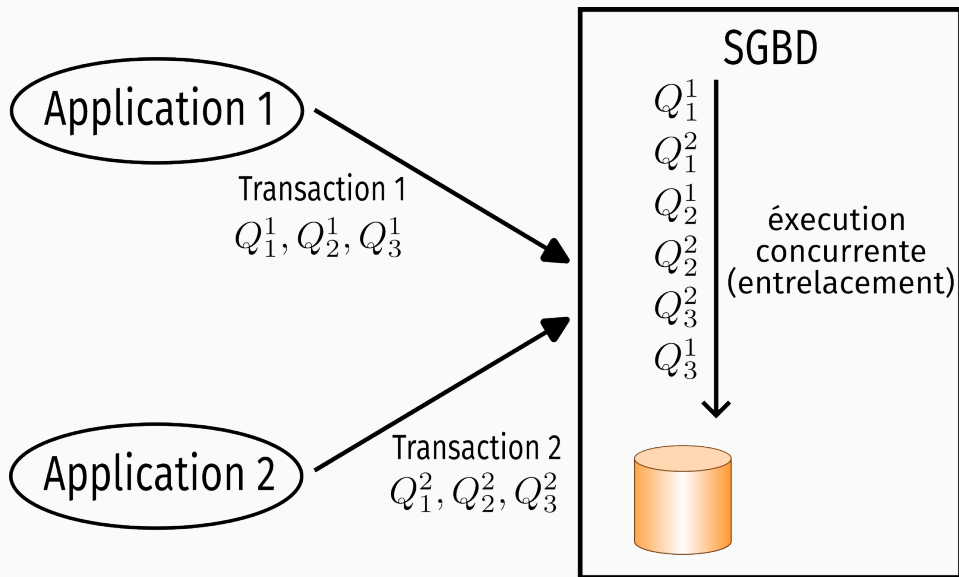
2. tentative d'exécution de la première requête

num	solde
123	100
456	300

3. erreur, la transaction est annulée

4. à la fin de la transaction, retour en arrière

num	solde
123	100
456	200



Pourquoi faire des exécutions concurrentes ?

- Les SGBD sont faits pour être utilisés par plusieurs dizaines, voir centaines d'applications en parallèle.
- Sans exécution concurrente, il faut bloquer les autres applications lors que l'exécution de la transaction d'une application (imaginez faire la queue à un unique guichet pour la SNCF).
- Les systèmes modernes ont plusieurs coeurs sur leur processeur ce qui permet de faire des exécutions concurrentes et donc de réduire le temps d'exécution de plusieurs transactions.

Définition

Les transactions doivent modifier la base comme si chacune était exécutée seule.

Pour les exécutions concurrentes

On veut que les exécutions concurrentes ait le **même effet** sur la base qu'une exécution **en série** des transactions. On dit d'une telle exécution qu'elle est **sérialisable**.

Exemple

On souhaite que l'exécution concurrente de la transaction 1 et 2 est le même effet sur la base que :

- soit l'exécution de la transaction 1 suivie de la transaction 2
- soit l'exécution de la transaction 2 suivie de la transaction 1

La cohérence est une conséquence :

- du principe de cohérence des transactions (une transaction complète et isolée laisse la base cohérente)
- de l'atomicité (une transaction est complète ou sans effet)
- et de l'isolation (les exécutions concurrentes ont l'effet une séquence de transactions)

Remarque

Les contraintes d'intégrité permettent de coder des transactions, qui ne vérifient pas toujours le principe de cohérence. En cas d'incohérence (violation de contrainte), le SGDB se charge d'annuler la transaction.

Révocabilité des opérations

- COMMIT et ROLLBACK sont irrévocables
- les écritures sont révocables tant que la transaction ne s'est pas terminée
- les écritures sont irrévocables une fois la transaction validée (après le commit)

Durabilité : l'effet des transactions validées est persistant

- stockés sur le disque dur
- persistant même si une panne arrive 1ms après la validation

Qu'est-ce qu'une transaction ?

1. Une opération d'écriture dans la base
2. Une opération suivie d'une opération de lecture
3. Une séquence d'opérations, lecture ou écriture, terminée par commit ou rollback
4. La séquence des opérations effectuées par un programme

J'exécute plusieurs fois de suite le même programme

1. J'obtiens toujours la même transaction
2. J'obtiens toujours la même séquence de transactions
3. À chaque exécution les transactions peuvent changer
4. Je n'obtiens jamais la même séquence de transactions

Que signifie « Atomicité »

1. Le SGDB effectue toutes les opérations en même temps
2. Le SGDB sait annuler ou valider solidairement toutes les opérations d'une même transaction
3. Le SGDB effectue et valide les opérations une par une

J'exécute la commande `DELETE * from Compte; where id=123;`

Réponse du serveur : 100 000 lignes détruites. Pourquoi et que faire ?

1. J'éteins tout et je pars en courant
2. J'interromps la connexion à la base
3. J'effectue un `ROLLBACK`
4. J'effectue un `COMMIT`

Que se passe-t-il en cas de panne au milieu d'une transaction ?

1. La transaction se finit automatiquement quand le SGBD redémarre
2. Toutes les mises à jour déjà effectuées sont annulées quand le SGBD redémarre
3. Le SGDB a un dispositif de sécurité pour toujours effectuer un **COMMIT** juste avant qu'une panne survienne

Exercice : Que faire dans cette situation pour conserver les propriétés ACID ?

Transfert de 100€

Comptes impossables

num	solde
123	80
456	50

```
UPDATE Compte SET solde = solde + 100
WHERE num = 456;
```

num	solde
123	80
456	150

```
SELECT num FROM Compte WHERE solde > 100
sauvegarde du résultat dans la base
COMMIT;
```

```
UPDATE Compte SET solde = solde - 100
WHERE num = 123; ❌
ROLLBACK; -- le solde est négatif
```

num	solde
123	80
456	50

Exécutions sérialisables

Description simplifiée d'une transaction

Une transaction est une liste ordonnée d'opérations parmi :

- `read(X,t)` lit le tuple `X` et stocke sa valeur dans la variable `t`.
- `write(X,t)` écrit la valeur de `t` dans le tuple `X`.
- `Commit` valide la transaction
- `Abort` annule la transaction

Exemples de deux transactions T_1 et T_2

- T_1 : `read(A,t); t = t - 100; write(A,t); read(B,t); t = t + 100; write(B,t);`
`Commit`
- T_2 : `read(A,t); t = 2 * t; write(A,t); read(B,t); t = 2 * t; write(B,t);`
`Commit`

Exécution

Une exécution des transactions $T_1, T_2, \dots, T_n$ est une liste d'opérations :

- contenant toutes les opérations des transactions T_i
- apparaissant dans le même ordre avec lequel elles apparaissent dans T_i

Exécution en série

Une exécution en série $(T_1, T_2, \dots, T_n)$ est l'exécution des opérations de T_1 , suivies de celles de T_2 , etc.

Exemple

Considérer l'effet des exécutions en série (T_1, T_2) et (T_2, T_1) sur la base initiale où $A = B = 125$.

Exécutions équivalentes

Deux exécutions des transactions $T_1, \dots, T_n$ sont équivalentes si elles ont le même effet sur toute base initiale, c'est à dire qu'elles produisent la même base finale.

Exécution sérialisable

Une exécution des transactions $T_1, \dots, T_n$ est sérialisable si elle est équivalente à une exécution en série des mêmes transactions (l'ordre des transactions dans la série pouvant être permuté).

Remarques

Pour garantir la propriété d'isolation, les SGBDs doivent s'assurer que l'exécution concurrente de leurs transactions en cours est sérialisable.

Le SGBD ne peut pas détecter toutes les exécutions sérialisables

Exemple d'exécution sérialisable

T_1

```
read(A,t)
t = t - 100
write(A,t)
```

T'_1

```
read(A,t)
t = t - 100
write(A,t)
read(B,t)
t = t + 100
write(B,t)
Commit
```

```
read(B,t)
t = t + 100
write(B,t)
Commit
```

- Le SGBD **voit** la même séquence d'opérations que dans l'exemple précédent non-sérialisable.
- Le SGBD ne peut donc pas détecter la sérialisabilité d'une exécution en général.
- On va introduire une notion moins générale mais reconnaissable, la **sérialisabilité par conflit**.

Nouvelle notation des opérations

Dans une exécution de $T_1, T_2, \dots, T_n$, on trouve les opérations :

- $r_i(X)$ une lecture de X dans T_i
- $w_i(X)$ une écriture dans X dans T_i
- c_i validation de T_i
- a_i annulation de T_i

Exercice

Traduire les exécutions vues précédemment à l'aide de cette nouvelle notation.

Remarque

Cette notation va nous permettre de repérer de manière automatisable les situations où des anomalies dues à la concurrence peuvent apparaître : les **conflits**.

Conflit WR, lecture impropre ou dirty read

Ce conflit arrive quand il y a une lecture, depuis une transaction, d'une donnée précédemment modifiée par une autre transaction non validée.

Avec la nouvelle notation, un conflit WR apparaît dans une exécution si il existe une opération $w_i(X)$ suivie plus loin de $r_j(X)$, avec $i \neq j$. On dit qu'il y a alors un conflit WR entre ces deux opérations.

Exercice

Trouver la lecture impropre dans l'exemple non sérialisable précédent.

Conflit RW ou lecture non répétable

Un conflit RW arrive quand il y a une écriture, depuis une transaction, d'une donnée précédemment lue dans une autre transaction. La transaction qui a effectué la lecture ne peut alors pas la reproduire.

Un conflit RW dans une exécution correspond une opération $r_i(X)$ suivie plus loin de $w_j(X)$ avec $i \neq j$.

Conflit WW ou mis à jour perdue

Un conflit WW arrive quand l'écriture d'une donnée depuis une transaction est effacée par l'écriture dans une autre transaction.

Un conflit WW dans une exécution correspond à une opération $w_i(X)$ suivie plus loin de $w_j(X)$ avec $i \neq j$.

Remarques

Chaque conflit peut être vu comme un blocage dans le réordonnancement possible des opérations au sein d'une exécution : inverser deux opérations en conflit peut changer son effet d'une exécution.

Sérialisabilité par conflit d'exécution de transactions validés

Exécutions équivalentes par conflit

Deux exécutions des transactions **validées** (terminant par un commit) $T_1, \dots, T_n$ sont équivalentes par conflit si on peut passer de l'une à l'autre en inversant successivement des opérations consécutives qui ne sont pas en conflit.

Exécution sérialisable par conflit

Une exécution des transactions validées $T_1, \dots, T_n$ est sérialisable par conflit si elle est équivalente par conflit à une exécution en série des mêmes transactions.

Propriété

Si une exécution sur des transactions validées est sérialisable par conflit, alors elle est sérialisable (tout court).

Remarques

En présence de transactions annulées, d'autres phénomènes peuvent arriver, nous verrons ça plus tard.

Graphe de précédence

Définition

Le graphe de précédence d'une exécution des transactions validées $T_1, \dots, T_n$ est un graphe orienté avec :

- un noeud pour chaque transaction T_i
- une arrête de T_i vers T_j dès qu'il existe une opération O_i de T_i en **conflit** avec une opération O_j de T_j telles que O_i est exécutée **avant** O_j dans E .

Exemple

$E = w1(A)r2(A)r1(A)w3(A)c1c2c3$



Graphe de précédence

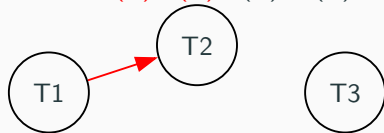
Définition

Le graphe de précédence d'une exécution des transactions validées $T_1, \dots, T_n$ est un graphe orienté avec :

- un noeud pour chaque transaction T_i
- une arrête de T_i vers T_j dès qu'il existe une opération O_i de T_i en **conflit** avec une opération O_j de T_j telles que O_i est exécutée **avant** O_j dans E .

Exemple

$E = w1(A)r2(A)r1(A)w3(A)c1c2c3$



Graphe de précédence

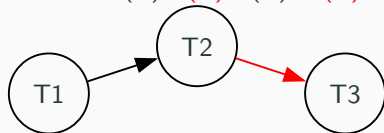
Définition

Le graphe de précédence d'une exécution des transactions validées $T_1, \dots, T_n$ est un graphe orienté avec :

- un noeud pour chaque transaction T_i
- une arrête de T_i vers T_j dès qu'il existe une opération O_i de T_i en **conflit** avec une opération O_j de T_j telles que O_i est exécutée **avant** O_j dans E .

Exemple

$E = w1(A)r2(A)r1(A)w3(A)c1c2c3$



Graphe de précédence

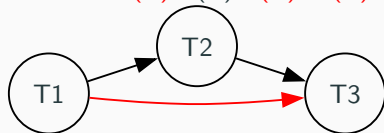
Définition

Le graphe de précédence d'une exécution des transactions validées $T_1, \dots, T_n$ est un graphe orienté avec :

- un noeud pour chaque transaction T_i
- une arrête de T_i vers T_j dès qu'il existe une opération O_i de T_i en **conflit** avec une opération O_j de T_j telles que O_i est exécutée **avant** O_j dans E .

Exemple

$E = w1(A)r2(A)r1(A)w3(A)c1c2c3$



Graphe de précédence

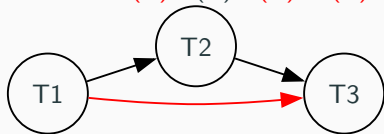
Définition

Le graphe de précédence d'une exécution des transactions validées $T_1, \dots, T_n$ est un graphe orienté avec :

- un noeud pour chaque transaction T_i
- une arrête de T_i vers T_j dès qu'il existe une opération O_i de T_i en **conflit** avec une opération O_j de T_j telles que O_i est exécutée **avant** O_j dans E .

Exemple

$E = w1(A)r2(A)r1(A)w3(A)c1c2c3$



Le graphe signifie qu'une exécution en série équivalente par conflit à E possède :

- $T1$ est exécutée avant $T2$
- $T2$ est exécutée avant $T3$
- $T1$ est exécutée avant $T3$

Donc E est équivalente en série à $(T1, T2, T3)$.

Propriétés du graphe de précédence

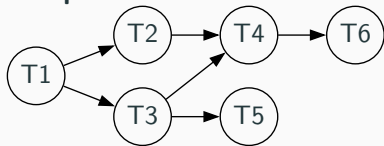
Décider la sérialisabilité par conflit

Une exécution de transactions validées est **sérialisable par conflit** si et seulement si son graphe de précédence est **acyclique**.

Trouver les exécutions en série équivalentes par conflit

Une exécution de transactions validées est sérialisable en toutes exécutions en série suivant un **tri topologique** de son graphe de précédence, i.e. où une transaction T_i est avant T_j dès qu'il existe un arc de T_i à T_j .

Exemple



- $(T1, T2, T3, T4, T5, T6)$
- trouver les autres

Tant que l'exécution ne contient que des transactions qui seront validées à la fin.

- Il suffit d'empêcher les conflits circulaires entre les transactions.
- Il est difficile d'empêcher les problèmes de sérialisabilité à l'aide du graphe de précédence (analyse a posteriori).

Est-ce que l'on peut appliquer ce que l'on a vu aux exécutions avec des annulations de transactions ?

Non ...

Annulation de transaction : réparabilité et journalisation

Sérialisabilité avec annulation : même définition, nouveaux problèmes

Exécution sérialisable (les transactions annulées n'ont pas d'effet)

Une exécution des transactions $T_1, \dots, T_n$ est sérialisable si elle est équivalente à (a même effet que) une exécution en série des mêmes transactions .

Exemple non sérialisable

T1	T2
r1(A)	
w1(A)	
	r2(A)
	w2(A)
	r2(B)
	w2(B)
	c2
a1	

Pire encore cette exécution est non-réparable, i.e. on ne peut pas “défaire” T2 au moment où T1 est annulée.

Exécution réparable

Définition

On dit qu'une exécution est *réparable* si pour chaque transaction T_i qui lit une ligne modifiée par une autre transaction T_j , T_i est validée uniquement après que T_j le soit.

Exemple réparable

T1	T2
r1(A)	
w1(A)	
	r2(A)
	w2(A)
	r2(B)
	w2(B)
a1	...

pas de commit, on peut annuler T2

Le SGBD va préparer cette exécution en annulant T2 aussi, car elle lit une modification de T1.

Annulation en cascade

T1	T2	T3	T4
r1(A)			
w1(A)			
	r2(A)		
	w2(A)		
		r3(A)	
		w3(C)	
	r2(B)		
	w2(B)		
			r4(C)
a1	...	...	...

Il n'y a pas d'autre choix que d'annuler T2, T3 et T4 après que T1 s'annule. Ce comportement est causé uniquement par les lectures impropres.

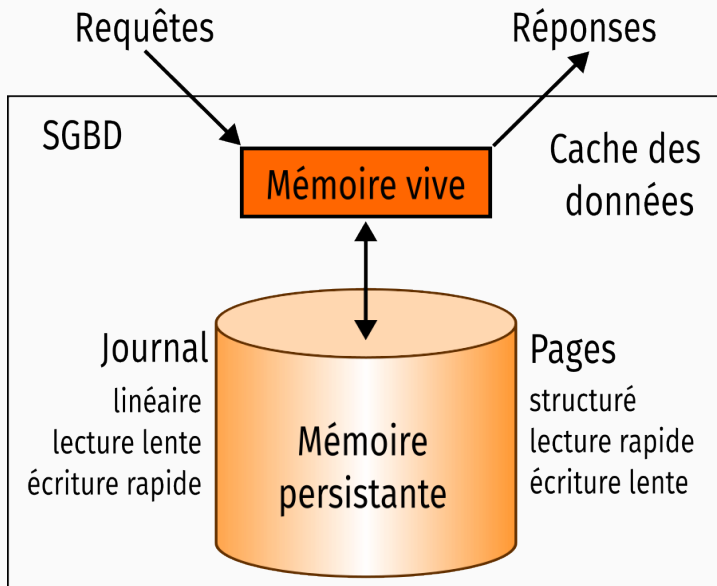
Définition

Une exécution est sans annulation en cascade si elle ne contient pas de lecture impropre : toute lecture lit une ligne modifiée par une transaction validée ou par la transaction auquel elle appartient.

Remarques

- Toute exécution sans annulation en cascade est réparable.
- La plupart des SGBDs interdisent les lectures impropres pour cette raison.

Facettes de la mémoire



Journal (log)

Il sert à assurer la durabilité et l'atomicité des transactions, il est notamment utile :

- lors de l'annulation d'une transaction,
- lors de la reprise après une panne système.

Le SGBD utilise le journal pour stocker :

- des informations sur les mises à jour de la base,
- les opérations de `COMMIT`,
- les points de sauvegarde,
- dans l'ordre dans leur évaluation.

Vocabulaire

- **point de sauvegarde (checkpoint)** : moment où toutes les opérations validées du journal ont été appliquées sur la base (les pages).
- **Image avant** : valeur d'une tuple avec qu'une mise à jour lui soit appliquée.

Processus de journalisation

- La journalisation des mises à jour et commit est effectués juste **avant** le changement d'état.
- Des points de sauvegardes sont effectués régulièrement par le SGDB

Transaction		Etat		Journal			
T_1 (plus 100)	T_2 (fois 2)	A	B	Trans.	Type	Tuple	Image avant
		25	50		POINT DE SAUVEGARDE		
write(A,125)		125		T_1	maj	A	25
	write(A,250)	250		T_2	maj	A	125
write(B, 150)			150	T_1	maj	B	50
	write(B, 300)		300	T_2	maj	B	150
Commit				T_1	commit		

Annulation d'une transaction

Défaire une transaction

- On lit le journal dans le sens inverse
- en appliquant les images avant de la transaction.

Transaction		Etat		Journal			
T_1 (plus 100)	T_2 (fois 2)	A	B	Trans.	Type	Tuple	Image avant
		25	50	POINT DE SAUVEGARDE			
write(A,125)		125		T_1	maj	A	25
	write(A,250)	250		T_2	maj	A	125
write(B, 150)			150	T_1	maj	B	50
	write(B, 300)		300	T_2	maj	B	150
Commit				T_1	commit		
	Rollback						
		125	150				

Reprise après panne

1. Identifier les transactions non validées depuis le dernier point de sauvegarde.
2. Défaire toutes les transactions non validées (en même temps).
3. Ajouter un point de sauvegarde.

Transaction		Etat		Journal			
T_1 (plus 100)	T_2 (fois 2)	A	B	Trans.	Type	Tuple	Image avant
		25	50				POINT DE SAUVEGARDE
write(A,125)		125		T_1	maj	A	25
	write(A,250)	250		T_2	maj	A	125
write(B, 150)			150	T_1	maj	B	50
	write(B, 300)		300	T_2	maj	B	150
Commit				T_1	commit		
PANNE ET REDÉMARRAGE							
			150				
		125					
POINT DE SAUVEGARDE							

Annulation avec écritures sales

Transaction		Etat		Journal			
T_1 (plus 100)	T_2 (fois 2)	A	B	Trans.	Type	Tuple	Image avant
		25	50	POINT DE SAUVEGARDE			
write(A,125)		125		T_1	maj	A	25
	write(A,250)	250		T_2	maj	A	125
write(B, 150)			150	T_1	maj	B	50
	write(B, 300)		300	T_2	maj	B	150
	Commit			T_2	commit		
Rollback			50				
		25					

- **écriture sale** : écriture d'un tuple modifié par une autre transaction non validée.
- En cas d'écriture sales, les modifications d'une transaction validé peuvent être écrasées par un rollback.

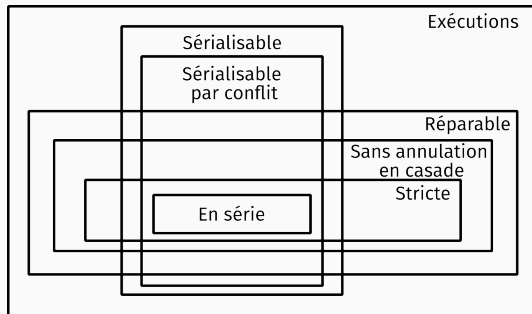
Exécution stricte

Il est souhaitable que le SGDB assure que ses exécutions soient :

- sans lecture sale
- sans écriture sale

Une telle exécution est dite **stricte**.

Cela vient s'ajouter au besoin de sérialisabilité, que nous avons étudié précédemment.



Verrous et 2PL stricte

Objectif

Il réordonne les opérations des différentes transactions par rapport à l'ordre dans lequel elles sont reçues.

Il peut

- **mettre en pause** les opérations d'une transaction
- **annuler une transaction** en cas de problème de sérialisabilisation

Système de verrous (locks)

Principe

Pour assurer la sérialibilité d'une exécution, nous allons conditionner les lectures et les écritures dans chaque transaction T_i à l'obtention de verrous :

- **un verrou partagé (shared lock)** $s_i(A)$ pour une lecture de A , $r_i(A)$
- **un verrou exclusif** $x_i(A)$ pour une écriture sur A , $w_i(A)$

Remarques

- Un verrou partagé n'est plus nécessaire dès lors qu'un verrou exclusif est déjà obtenu.
- Un verrou sur A peut être relâché (unlocked) au cours d'une transaction T_i avec $u_i(A)$.

Transactions bien formées

Dans une exécution, toutes les transactions vérifient :

1. un tuple est verrouillé de manière partagé (resp. exclusive) par la transaction au moment où il est lu (resp. écrit).
2. avant qu'elle ne se termine, tous les verrous d'une transaction sont relâchés.

Exécution légale

À tout moment d'une exécution légale, un tuple est :

- soit verrouillé de manière exclusive par une seule transaction
- soit verrouillé de manière partagé par une ou plusieurs transactions
- soit non verrouillé

Exemple d'utilisation des verrous

T_1	T_2
$s_1(A)$	
$r_1(A)$	
$x_1(A)$	
$w_1(A)$	
	$s_2(A)$
	$r_2(A)$
	$x_2(A)$
	$w_2(A)$
	$u_2(A)$
$u_1(A)$	
Commit	
	Commit

Transactions bien formées .

Exemple d'utilisation des verrous

T_1	T_2
$s_1(A)$	
$r_1(A)$	
$x_1(A)$	
$w_1(A)$	
$u_1(A)$	
	$s_2(A)$
	$r_2(A)$
	$x_2(A)$
	$w_2(A)$
	$u_2(A)$
Commit	
	Commit

Transactions bien formées et l'exécution est légale.

Le protocole Two-Phase Locking (2PL)

Définition

Dans chaque transaction, tous les verrous sont obtenus avant que le premier verrou ne soit relâché.

Remarque

Dans une transaction, on obtient donc deux phases :

1. une phase obtention des verrous,
2. une phase de relâchement des verrous.

Propriété

Une exécution légale de transactions bien formées qui suit le protocole 2PL est sérialisable par conflit.

Le protocole Strict Two-Phase Locking (Strict 2PL)

Définition

Dans chaque transaction, les verrous sont uniquement (et tous) relâchés au moment où la transaction se termine.

Propriété

Une exécution légale de transactions bien formées qui suit le protocole 2PL strict est strict.

Les interblocages (deadlocks)

Exemple du transfert bancaire et du taux

T_1

T_2

$s_1(A)$

$r_1(A)$

$s_2(B)$

$r_2(B)$

En attente de $s_2(B)$ pour la suite

$(x_1(B))$

$(w_1(B))$

En attente de $s_1(A)$ pour la suite

$(x_2(A))$

$(w_2(A))$

Avec le protocole 2PL, il peut arriver que plusieurs transactions attendent mutuellement un verrou déposé par les autres, créant une attente infinie.

- Peu de deadlock (moins de 1% des transactions)
- Les SGDB utilisent des systèmes de verrou plus complexes, qui limitent d'avantage les cas de deadlock.
- Le surcoût des verrous vient principalement des transactions mis en attente.

Isolation en pratique

En SQL, il existe plusieurs niveaux d'isolation, du plus permissif au plus strict
Plus le niveau est permissif, plus l'exécution est fluide, plus les anomalies sont possibles.

Plus le niveau est strict, plus l'exécution risque de rencontrer des blocages, moins les anomalies sont possibles.

Le niveau d'isolation par défaut n'est jamais le plus strict

- Pas forcément utile dans la grande majorité des cas,
- provoque rejets et blocages difficiles à expliquer à l'utilisateur.

Pour que l'isolation soit totale, il faut l'indiquer explicitement

Niveau d'isolation d'une transaction en SQL

Définis en fonction des anomalies possibles :

Niveau d'isolation	lect. impropres	lect. non reproductibles	tuples fantômes et anomalies
READ COMMITTED	impossible	possible	possible
REPEATABLE READ	impossible	possible	possible
SERIALIZABLE	impossible	impossible	impossible

En pratique

On peut définir le niveau d'une transaction de la manière suivante :

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

Dans Oracle et Postgres, le niveau d'isolation par défaut est READ COMMITTED.

Tuples fantômes

Définition

Tuples qui apparaissent (ou disparaissent) d'une exécution à une autre des résultats d'une requête qui effectue un filtre sur les données.

Attention : Il ne s'agit pas d'une lecture non reproductible, les valeurs utilisées pour filtrer ont changées mais pas forcément les données lues initialement.

Exemple

Le résultat de cette requête peut changer si des produits sont insérés

```
SELECT id FROM Produit WHERE prix > 4;
```

- Le protocole 2PL ne permet pas d'éviter ce type de phénomène, car il suppose que les lectures se font par identifiant de tuples et non par filtre.
- Dans les SGBDs, ces anomalies peuvent être évitées par des **verrous de prédicat** (verrous sur la condition WHERE).

À retenir

Une requête accède à l'état de la base **au moment où la requête est exécutée**.

Pas de lecture sale, car donnée en cours de modification ne fait pas partie de l'état de la base.

Beaucoup d'anomalies, mais assez fluide.

À retenir

Une requête accède à l'état de la base **au moment où la transaction commence**.

Pas de lecture sale, pas de lecture non répétable : les requêtes accèdent toujours au même état de la base.

Les mises à jour perdues sont possibles. (pas dans Postgres)

Verrous explicites : la clause for update

On souhaite lire des données pour les modifier ensuite.

- Le système ne peut pas le deviner
- Contrairement à la s rialisabilit , pas besoin de tracer toutes les lectures
- Le syst me pose un verrou exclusive sur les donn es
- Son utilisation est laiss  dans les mains du d veloppeur (facteur humain)

Snapshot isolation

Définition

Snapshot isolation est une gestion de la concurrence telle que :

- chaque transaction travaille sur une “copie” initié par l'état de la base au début de la transaction
- une transaction peut être validée seulement si aucune modification concurrente n'a été validée depuis le début de la transaction, sinon elle est annulée.

Propriétés

- Les lectures ne sont ni bloquées, ni bloquantes pour les écritures concurrentes.
- Snapshot isolation ne garantit pas la sérialisabilité, il peut y avoir des pertes de mises à jour
- Snapshot isolation assure qu'il n'y a pas de lecture sale, pas d'écriture sale, pas de lecture non reproductible et pas de tuple fantôme.

En pratique (notamment Postgres)

- elle est utilisé pour assurer le niveau d'isolation REPEATABLE READ.